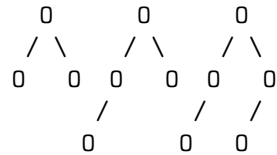


Homework 3

Problem 1 - Weiss 4.6

Let N be the number of full nodes and L be the number of leaves.

Base Case: A tree with 1 full node ($N = 1$) has 2 leaves ($L = 2$)



These are the only three possible cases for $N = 1$.

Assumption: Assume that a tree with $N = n$ full nodes has $L = n + 1$ leaves.

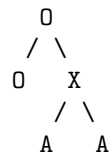
Inductive Step: Prove for $N = n + 1$ $L = (n + 1) + 1$.

In order for our theorem to be proven right the two sides of the following formula need to be equivalent.

$$\underbrace{(n + 1) + 1}_{\text{Lefthand Side}} = \underbrace{n + 1 + Y}_{\text{Righthand Side}}$$

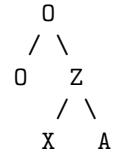
- Lefthand Side: Number of leaves a tree with $n+1$ full nodes has.
- Righthand Side: Number of leaves a tree with n nodes has plus Y .
- Y : Number of leaves needed to be added to a tree with n full nodes to make it have $n + 1$ full nodes.

In order to add an additional full node to the tree (i.e. convert a leaf to a full node), we can (1) add two leaves to a node that was once a leaf.



In the diagram above, X and 0s are pre-existing node and As are newly added. In order to make the leaf X a full node, we have to add two children (As) to X. This causes the number of leaves to reduce by one (since X is no longer a leaf) and then increase by 2 (since we added 2 leaves). Hence, $Y = 2 - 1$

We can also (2) add a leaf to a node that has one child.



Again, Xs, Zs and 0s are pre-existing nodes. We add A as a leaf. This also increases the number of leaves by 1 in total. Note that Z was not a leaf node. Hence, $Y = 1$

These two cases cover all the possible cases of adding a full node to a tree.

$$(n + 1) + 1 = n + 1 + Y$$

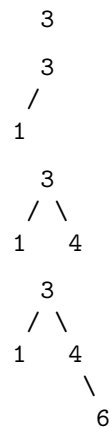
$$(n + 1) + 1 = n + 1 + (1)$$

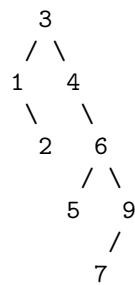
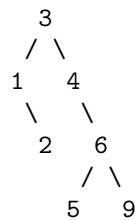
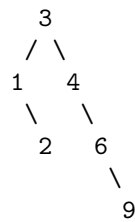
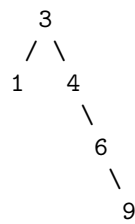
$$n + 2 = n + 2$$

We have thus proven the inductive step. Since the base case is proven and the inductive step is proven, we have an inductive proof.

Problem 2 - Weiss 4.9

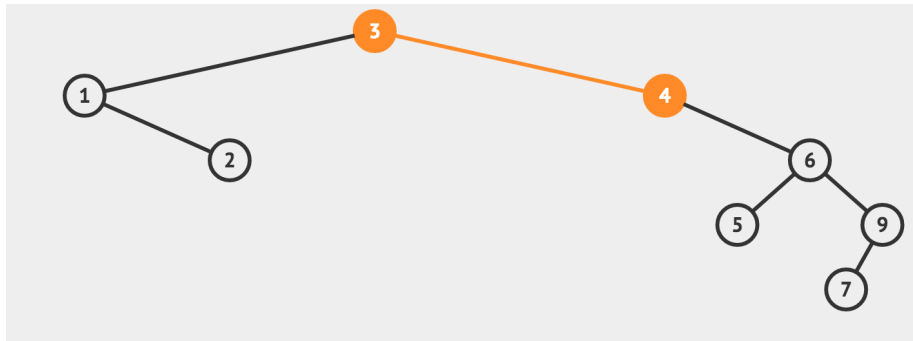
(a) Inserting 3, 1, 4, 6, 9, 2, 5, 7 into a BST



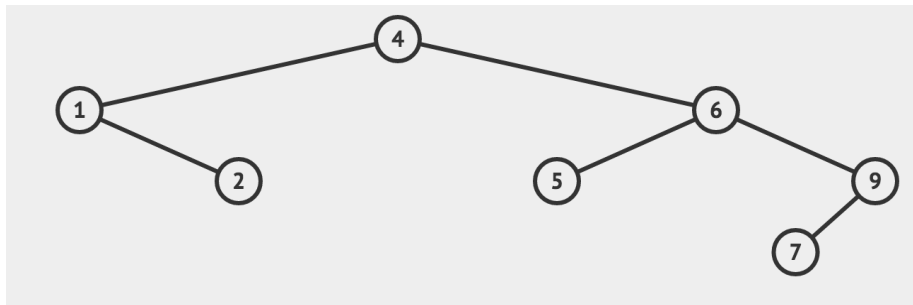


(b) Deleting the root

We find the minimum on the right subtree of the root and replace root with that element. Here, we replace 3 with 4.

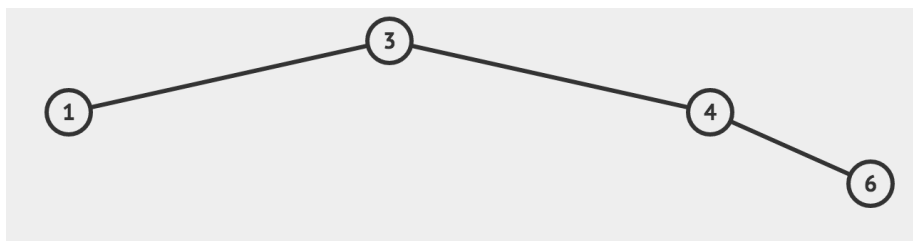


We then remove 4, which is done by making 6 (which is the only child of 4) the right child of the new root 4.

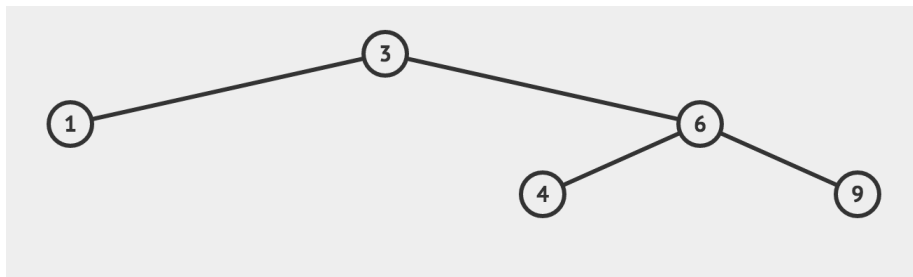
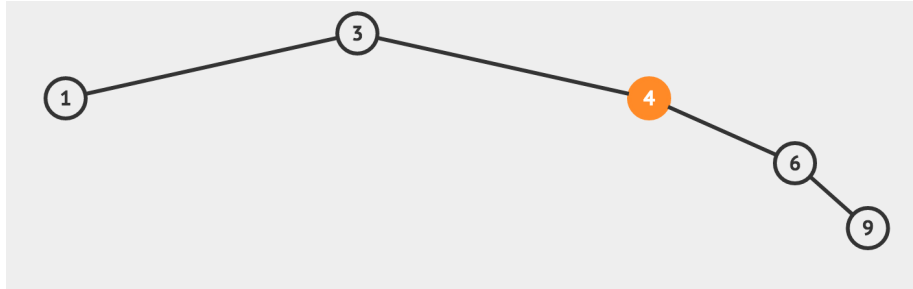


Problem 3 - Weiss 4.9 (AVL Version)

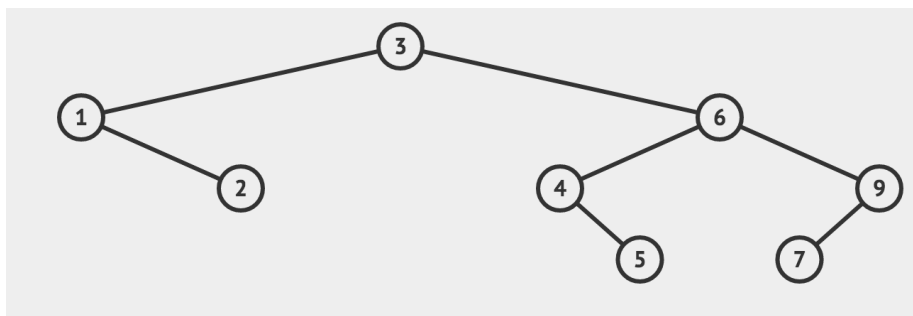
1. Insert 3: no rotation needed since left and right subtrees are both 0
2. Insert 1: no rotation needed. (1 left height: 0, right height: 0) (3 left height: 1, right height: 0). Pro tip: this is how you should check for imbalances – by counting the left and right subtree heights starting from the bottom.
3. Insert 4: no rotation needed.
4. Insert 6: no rotation needed.



5. Insert 9: single rotation needed at 4.

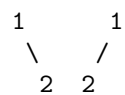


- 6. Inserting 2: no rotation needed
- 7. Inserting 5: no rotation needed
- 8. Inserting 7: no rotation needed



Problem 4

Consider these two trees:



They both have the following preorder and postorder traversals:

- Preorder: [1, 2]
- Postorder: [2, 1]

Hence, having only the preorder and postorder traversal output does not reconstruct trees.

However, if we add an additional constraint: that no node can have a right child without having a left child (or equivalently, that the children of nodes are symmetric), then preorder and postorder do uniquely determine a tree.

For example, say we have the following traversals:

- Preorder: [4, 2, 1, 3, 5, 6]
- Postorder: [1, 3, 2, 6, 5, 4]

Since preorder goes **Self, Left, Right** and postorder goes **Left, Right, Self**, we know that 4 is the root of the tree. This is because 4 is first in the preorder traversal, and since we start from the root of the tree, the root (*self* in that step of the recursion) will be printed first by the preorder.

We also know that 2 is the root of the left subtree. This is from the preorder traversal and from our condition: since the preorder traversal prints *left* after *self* (self was 4), and that 4 cannot have a right child without having a left child first, and that 4 must have at least 1 child (if not the tree won't be able to continue), then 2 must be the left child of 4. Hence, 2 is the root of the left subtree.

We further know that 5 is the root of the right subtree. This is from the postorder traversal. Since the postorder traversal goes **Left, Right, Self**, the node immediately before *self* would be *right*. How do we know that 5 is not on the left? Because we know for sure that 2 is the root of the left subtree. Hence, if 4 only had a left subtree, the sequence would end with [... 2, 4]. However, since it ends with [... 2 ... 5, 4], the portion ... 5 would be part of the postorder traversal of the right subtree, which eventually comes back to 5, then the root 4.

Hence, we have the structure of two subtrees:

- Root: 4
- Left subtree:
 - Preorder: [2, 1, 3]
 - Postorder: [1, 3, 2]
- Right subtree:
 - Preorder: [5, 6]
 - Postorder: [6, 5]

Each of the subtrees can be solved recursively using the same analysis.

Hence, a tree can be uniquely determined using preorder and postorder traversals **if we impose the restriction on a node's children.**

Problem 5

Full code in BSTLazy.java. In particular, we modify the node by adding a boolean deleted.

```
public class NodeLazy<Q extends Comparable<? super Q>> {
    public Q data;
    public NodeLazy<Q> left;
    public NodeLazy<Q> right;
    public boolean deleted = false;

    public NodeLazy(Q data, NodeLazy<Q> left, NodeLazy<Q> right) {
        this.data = data;
        this.left = left;
        this.right = right;
    }

    public String toString() {
        return data.toString();
    }
}
```

Then for findMin, we do the following:

```
public T findMin() {
    NodeLazy<T> node = findMin(root);
    return node != null ? node.data : null;
}

private NodeLazy<T> findMin(NodeLazy<T> node) {
    if (node == null) {
        return null;
    }

    NodeLazy<T> leftMin = findMin(node.left);

    if (leftMin != null) {
        // left minimum node exists
        return leftMin;
    }
    // now the left minimum node is null
    // we check if current node is deleted.
    // if it is, we need to search right subtree
    // if not, we are at the minimum
    if (node.deleted) {
        return findMin(node.right);
    } else {

```

```

    return node;
}
}

```

The idea is that when we are at a deleted node, we would first look for a minimum in the node's left subtree. If there is such a minimum, we return it. Otherwise, we need to look in the deleted node's **right** subtree for another minimum candidate.

If the node isn't deleted, we proceed as normal – keep descending via the left child.

There's a public `findMin()` and a private recursive `findMin` that takes in a `NodeLazy<T>` as parameter. If the `node` is `null`, we return `null`. Otherwise, we recursively go one level deeper by trying to find the minimum in the left subtree of the current `node` in the line `NodeLazy<T> leftMin = findMin(node.left);`. If the returned minimum of the left subtree is not `null`, we have successfully found a minimum. We return it. Otherwise (ie. `leftMin == null` is `true`), we know that the left subtree does not contain a minimum. We check if the current `node` is deleted. If it is not deleted, then the current `node` will be the minimum. Otherwise, if it is deleted, it is ineligible for being the minimum. Hence, we are left with the option of searching the right subtree which we do in the line `findMin(node.right);`